¿Qué es un Array?

Antes de adentrarnos en los métodos, recordemos que un **array** es una estructura de datos que nos permite almacenar una colección ordenada de elementos. Piensen en él como una lista donde cada elemento tiene una posición (o índice) numérica.

```
let frutas = ["Manzana", "Banana", "Cereza", "Mango"];
```

Métodos de Arrays comunes

Hoy profundizaremos en `forEach`, `map`, `filter`, `reduce`, y los métodos para modificar arrays como `push/pop/shift/unshift`, y `splice/slice`. También aclararemos la diferencia de eficiencia con `some()` y `find()`.

1. `forEach()`: Iteración Simple

El método `forEach()` se utiliza para **iterar** sobre cada elemento de un array. Es perfecto cuando queremos ejecutar una función para cada elemento, pero no necesitamos crear un nuevo array ni modificar el original.

Sintaxis:

```
array.forEach(function(elemento, indice, arrayOriginal) {
  // Código a ejecutar por cada elemento
});
```

- `elemento`: El elemento actual que se está procesando en el array.
- `indice` (opcional): El índice del elemento actual.
- `arrayOriginal` (opcional): El array sobre el que `forEach()` fue llamado.

Ejemplo:

Queremos imprimir cada fruta de nuestro array:

```
let frutas = ["Manzana", "Banana", "Cereza", "Mango"];

console.log("--- Usando forEach ---");

frutas.forEach(function(fruta) {
  console.log(`¡Me encanta comer ${fruta}!`);
});
```

2. `map()`: Transformación y Nuevo Array

El método `map()` es ideal cuando queremos **transformar** cada elemento de un array y crear un **nuevo array** con los resultados de esa transformación. Es importante recordar que `map()` siempre devuelve un nuevo array y no modifica el original.

Sintaxis:

```
let nuevoArray = array.map(function(elemento, indice, arrayOriginal)
{
  // Retorna el nuevo elemento transformado
});
```

Ejemplo:

Queremos crear un nuevo array con el nombre de cada fruta en mayúsculas:

```
let frutas = ["Manzana", "Banana", "Cereza", "Mango"];

console.log("\n--- Usando map ---");

let frutasEnMayusculas = frutas.map(function(fruta) {
  return fruta.toUpperCase();
});

console.log("Frutas originales:", frutas);
console.log("Frutas en mayúsculas:", frutasEnMayusculas);
```

3. `filter()`: Filtrado de Elementos

El método `filter()` se utiliza para **filtrar** elementos de un array y crear un **nuevo array** que contenga solo los elementos que cumplen con una condición específica. Al igual que `map()`, `filter()` no modifica el array original.

Sintaxis:

```
let nuevoArrayFiltrado = array.filter(function(elemento, indice,
arrayOriginal) {
  // Retorna true si el elemento debe incluirse en el nuevo array, false en caso contrario
});
```

Ejemplo:

Queremos obtener solo las frutas cuyo nombre tiene más de 6 letras:

```
let frutas = ["Manzana", "Banana", "Cereza", "Mango"];

console.log("\n--- Usando filter ---");
let frutasLargas = frutas.filter(function(fruta) {
  return fruta.length > 6;
});
console.log("Frutas originales:", frutas);
console.log("Frutas con más de 6 letras:", frutasLargas);
```

4. `reduce()`: Acumulación de un Solo Valor

El método `reduce()` es increíblemente versátil y se utiliza para **acumular un solo valor** a partir de los elementos de un array. Puedes sumar, concatenar, o incluso transformar un array en un objeto.

Sintaxis:

```
let valorAcumulado = array.reduce(function(acumulador, elemento,
indice, arrayOriginal) {
  // Lógica para acumular
  return nuevoAcumulador;
}, valorInicial); // valorInicial es opcional
```

- `acumulador`: El valor que se va acumulando en cada iteración.
- `elemento`: El elemento actual que se está procesando.
- `valorInicial` (opcional): El valor con el que comienza el `acumulador`. Si no se proporciona, el primer elemento del array se usa como `valorInicial` y la iteración comienza desde el segundo elemento.

Ejemplo:

Queremos calcular el precio total de una lista de productos:

```
const productos = [
  { nombre: "Laptop", precio: 1200 },
  { nombre: "Teclado", precio: 80 },
  { nombre: "Mouse", precio: 35 },
];

console.log("\n--- Usando reduce ---");

let precioTotal = productos.reduce(function(acumulador, producto) {
  return acumulador + producto.precio;
}, 0); // El 0 es el valor inicial del acumulador

console.log("Precio total de los productos:", precioTotal);
```

5. `push()` / `pop()` / `shift()` / `unshift()`: Modificando Extremos

Estos métodos **modifican el array original** añadiendo o eliminando elementos de sus extremos.

`push()`: Agrega uno o más elementos al **final** del array y devuelve la nueva longitud del array.

`pop()`: Elimina el **último** elemento del array y lo devuelve.

`shift()`: Elimina el **primer** elemento del array y lo devuelve.

`unshift()`: Agrega uno o más elementos al **principio** del array y devuelve la nueva longitud del array.

Ejemplo:

```
let tareas = ["Estudiar", "Hacer ejercicio", "Cocinar"];

console.log("\n--- Usando push/pop/shift/unshift ---");

console.log("Tareas originales:", tareas);

tareas.push("Comprar víveres");

console.log("Después de push 'Comprar víveres':", tareas);

let ultimaTarea = tareas.pop();

console.log("Después de pop (última tarea eliminada: " + ultimaTarea
+ "):", tareas);

let primeraTarea = tareas.shift();

console.log("Después de shift (primera tarea eliminada: " +
primeraTarea + "):", tareas);

tareas.unshift("Organizar escritorio");

console.log("Después de unshift 'Organizar escritorio':", tareas);
```

6. `splice()` / `slice()`: Modificando o Extrayendo Partes

Estos métodos son poderosos para manipular segmentos de un array.

`splice()`: **Modifica el contenido** de un array eliminando elementos existentes y/o agregando nuevos elementos en un lugar específico. **Modifica el array original**.

`slice()`: **Extrae una porción** de un array en un **nuevo array**, sin modificar el array original.

Sintaxis `splice()`:

```
array.splice(indiceInicio, cantidadAEliminar, elemento1,
elemento2, ...);
```

Sintaxis `slice()`:

```
let nuevoArray = array.slice(indiceInicio, indiceFin); // indiceFin
es exclusivo
```

Ejemplo:

```
let colores = ["Rojo", "Verde", "Azul", "Amarillo", "Blanco"];

console.log("\n--- Usando splice/slice ---");

console.log("Colores originales:", colores);

// Usando splice para eliminar y añadir
let coloresEliminados = colores.splice(1, 2, "Naranja", "Violeta");

// Elimina "Verde" y "Azul", añade "Naranja" y "Violeta"
console.log("Después de splice (eliminados: " + coloresEliminados +
"):", colores);

// Usando slice para extraer
let coloresPrimarios = colores.slice(0, 3); // Extrae los 3 primeros
console.log("Colores primarios (con slice):", coloresPrimarios);
console.log("Colores originales (después de slice):", colores); // El
original no cambia
```

`some()` y `find()`: Eficiencia para Casos Específicos

Si tu objetivo no es crear un nuevo array con todos los elementos que cumplen una condición, sino algo más específico, estos métodos pueden ser más eficientes que `filter`.

`some()`: **Devuelve `true` si al menos un elemento** en el array cumple con la función de prueba proporcionada, y `false` en caso contrario. Deja de iterar tan pronto como encuentra un elemento que satisface la condición.

```
let numeros = [10, 25, 30, 45, 50];
let hayNumeroMayorDe40 = numeros.some(function(num) {
  return num > 40;
});
console.log("\n--- Usando some ---");
console.log("¿Hay algún número mayor de 40?", hayNumeroMayorDe40);
// true
```

`find()`: **Devuelve el valor del primer elemento** en el array que cumple la función de prueba proporcionada. Si ningún elemento cumple la condición, devuelve `undefined`. También deja de iterar después de encontrar el primer elemento.

```
let usuarios = [
  { id: 1, nombre: "Ana" },
  { id: 2, nombre: "Juan" },
  { id: 3, nombre: "Pedro" },
];
```

```
let usuarioJuan = usuarios.find(function(user) {
  return user.nombre === "Juan";
});

console.log("\n--- Usando find ---");
console.log("Usuario encontrado:", usuarioJuan);
// { id: 2, nombre: "Juan" }
```

Método	Propósito Principal	¿Modifica el Array Original?	¿Devuelve un Nuevo Array?	Casos de Uso Comunes
forEach	Iterar sobre cada elemento	No	No (devuelve undefined)	Imprimir elementos, ejecutar una acción por cada elemento, depuración.
map	Transformar cada elemento y crear un nuevo array	No	Sí	Convertir datos, formatear elementos, escalar valores.
filter	Filtrar elementos basados en una condición	No	Sí	Obtener subconjuntos de datos, buscar elementos específicos, eliminar elementos no deseados.
reduce	Acumular un solo valor (suma, concatenación, objeto, etc.)	No	No (devuelve el valor acumulado)	Calcular totales, contar ocurrencias, aplanar arrays, agrupar datos.
push/pop	Añadir/eliminar elementos al/del final	Sí	No (devuelve longitud/elemento)	Manejo de pilas (stacks).
shift/unshift	Eliminar/añadir elementos al/del principio	Sí	No (devuelve elemento/longitud)	Manejo de colas (queues).
splice	Modificar contenido: eliminar/agregar/reemplazar en cualquier posición	Sí	No (devuelve elementos eliminados)	Insertar, eliminar o reemplazar elementos en medio de un array.
slice	Extraer una porción de un array en un nuevo array	No	Sí	Crear copias de arrays, obtener sub-arrays.
some	Verificar si al menos un elemento cumple una condición (devuelve boolean)	No	No (devuelve boolean)	Comprobar existencia de un elemento, validación rápida.
find	Obtener el primer elemento que cumple una condición	No	No (devuelve el elemento o undefined)	Buscar un elemento específico, obtener el primer match.

¿En Qué Casos Debo Usar Uno u Otro?
La elección del método adecuado depende de lo que necesites lograr:

- **Usa forEach() cuando:**

Simplemente quieres recorrer el array y realizar una acción con cada elemento, sin necesidad de un resultado específico (por ejemplo, imprimir en consola, enviar datos a una API).

No necesitas un nuevo array como salida.

- **Usa map() cuando:**

Necesitas transformar cada elemento del array en un nuevo formato o valor.

El resultado deseado es un nuevo array con los elementos transformados, y el array original debe permanecer intacto.

Ejemplo: Convertir una lista de números en sus cuadrados, obtener solo los nombres de una lista de objetos de usuario.

- **Usa filter() cuando:**

Necesitas seleccionar un subconjunto de elementos de un array que cumplen con una o más condiciones.

El resultado deseado es un nuevo array que contiene solo los elementos que pasaron la prueba, y el array original debe permanecer intacto.

Ejemplo: Obtener solo los usuarios activos, encontrar números pares, filtrar productos por precio.

- **Usa reduce() cuando:**

Necesitas consolidar todos los elementos del array en un único valor (un número, una cadena, un objeto, etc.).

Puedes usarlo para sumar, promediar, contar ocurrencias, agrupar objetos, o transformar un array en un objeto.

- **Usa push() / pop() / shift() / unshift() cuando:**

Necesitas añadir o quitar elementos rápidamente de los extremos de tu array y no te importa modificar el original. Son eficientes para manejar estructuras de datos tipo pila o cola.

- **Usa splice() cuando:**

Necesitas modificar el array original insertando, eliminando o reemplazando elementos en cualquier posición.

- **Usa slice() cuando:**

Necesitas obtener una copia parcial o total de un array sin modificar el original. Es excelente para crear sub-arrays.

- **Usa some() cuando:**

Solo necesitas saber si al menos un elemento cumple una condición, y no necesitas saber cuáles son o devolverlos. Es más eficiente que filter para esta tarea.

- **Usa find() cuando:**

Solo necesitas obtener el primer elemento que cumple una condición, y no te interesan los demás si los hay. También es más eficiente que filter si solo buscas un elemento.

¡Challenge !

¡Es hora de poner en práctica lo aprendido! Aquí les presento un desafío actualizado para afianzar sus conocimientos:

Escenario:

Tienes un array de objetos que representan productos en una tienda online. Cada objeto tiene las siguientes propiedades: nombre (string), precio (number), y disponible (boolean).

JavaScript

```
const productos = [
  { nombre: "Laptop Gamer", precio: 1200, disponible: true },
  { nombre: "Teclado Mecánico", precio: 80, disponible: false },
  { nombre: "Mouse Inalámbrico", precio: 35, disponible: true },
  { nombre: "Monitor UltraWide", precio: 450, disponible: true },
  { nombre: "Auriculares Bluetooth", precio: 60, disponible: false },
  { nombre: "Webcam HD", precio: 40, disponible: true },
];
```

Tu Tarea:

1. **Imprimir Productos:** imprimir en la consola el nombre y el precio de cada producto, en el formato: "Producto: [Nombre], Precio: $[Precio]".

2. **Productos en Oferta:** crear un nuevo array donde cada producto tenga una nueva propiedad llamada precioConDescuento que sea el 10% menos del precio original, solo para los productos que están disponible: true. Los productos no disponibles deben mantener su precio original en la nueva propiedad precioConDescuento.

3. **Productos Disponibles y Baratos:** obtener un nuevo array que contenga solo los productos que están disponible: true Y cuyo precio sea menor o igual a $100.

4. **Valor Total del Inventario:** calcular la suma total de los precios de todos los productos en el array original.

5. **Encontrar un Producto Específico:** encontrar el objeto completo del producto con el nombre "Webcam HD". Imprime el objeto encontrado o undefined si no existe.

¡Tómense su tiempo, experimenten y no duden en preguntar si tienen alguna duda! ¡Éxito!